

Omnichore

Technical Architecture & Solution Presentation

Lead System Architect Presentation

Version 1.0

Date: June 2026

Classification: Client-Facing / Solutions Architecture

Table of Contents

- Omnichore — Technical Architecture & Solution Presentation 1**
 - Executive Summary 1
 - 1. Architecture & System Design 1
 - 1.1 High-Level Architecture 1
 - 1.2 System Design Diagram 1
 - 1.3 DevOps Pipeline & Deployment Strategy 3
 - 1.4 User Journey: Video-to-Clip Pipeline 5
 - 2. Technology Stack & Microservices Breakdown 7
 - 2.1 Complete Technology Matrix 7
 - 2.2 Microservices Deep Dive 9
 - 2.3 Frontend Architecture 14
 - 3. AI Integration & Logic 14
 - 3.1 AI System Design 14
 - 3.2 AI Balance System — Weighted Virality Scoring 15
 - 3.3 AI Prompt Engineering System 16
 - 3.4 Data Communication Pipeline 17
 - 3.5 Credit-Based AI Consumption Model 17
 - 4. Enterprise-Grade Standards & Compliance 18
 - 4.1 Security Architecture 18
 - 4.2 Scalability Design 19
 - 4.3 Observability & Reliability 20
 - 4.4 Testing Strategy 22
 - 4.5 Data Privacy & Ethical AI 22
 - 4.6 Compliance-Ready Architecture 23
 - Appendix A: Repository Structure 23
 - Appendix B: Port Allocation Map 24

Omnichore — Technical Architecture & Solution Presentation

Prepared by: Lead System Architect
Classification: Non-Confidential — Stakeholder Distribution
Date: June 2026

Executive Summary

Omnichore is an enterprise-grade, distributed media processing and video-on-demand platform engineered from the ground up in **Rust**. The system enables end users to ingest long-form video content, automatically generate AI-powered transcripts, intelligently segment the content into viral-ready short-form clips, apply professional-grade subtitle styling and watermarking, and stream finished assets on demand — all orchestrated through a fully automated CI/CD pipeline deploying to hardened cloud infrastructure.

This document presents the engineering decisions, system design, AI integration strategy, and enterprise-grade standards that underpin Omnichore — demonstrating our capacity to deliver secure, scalable, production-ready software solutions.

1. Architecture & System Design

1.1 High-Level Architecture

Omnichore employs a hybrid of **Hexagonal Architecture (Ports & Adapters)** and a **Pyramid Architecture** across a Rust-native microservices ecosystem. This combination provides clean dependency boundaries, technology-agnostic core logic, and a clearly stratified request flow from ingress to worker execution.

Architectural Principles

Principle	Implementation
Hexagonal Architecture	Domain logic is decoupled from HTTP frameworks and databases via <code>api</code> , <code>domain</code> , and <code>driven</code> adapter layers
Pyramid Architecture	Traffic flows from Gateway → Orchestrator → Specialized Workers
Async-First	All services built on <code>tokio</code> async runtime with non-blocking I/O
Static Compilation	All binaries compiled to <code>aarch64-unknown-linux-gnu</code> with <code>+crt-static</code> for zero-dependency deployment

1.2 System Design Diagram

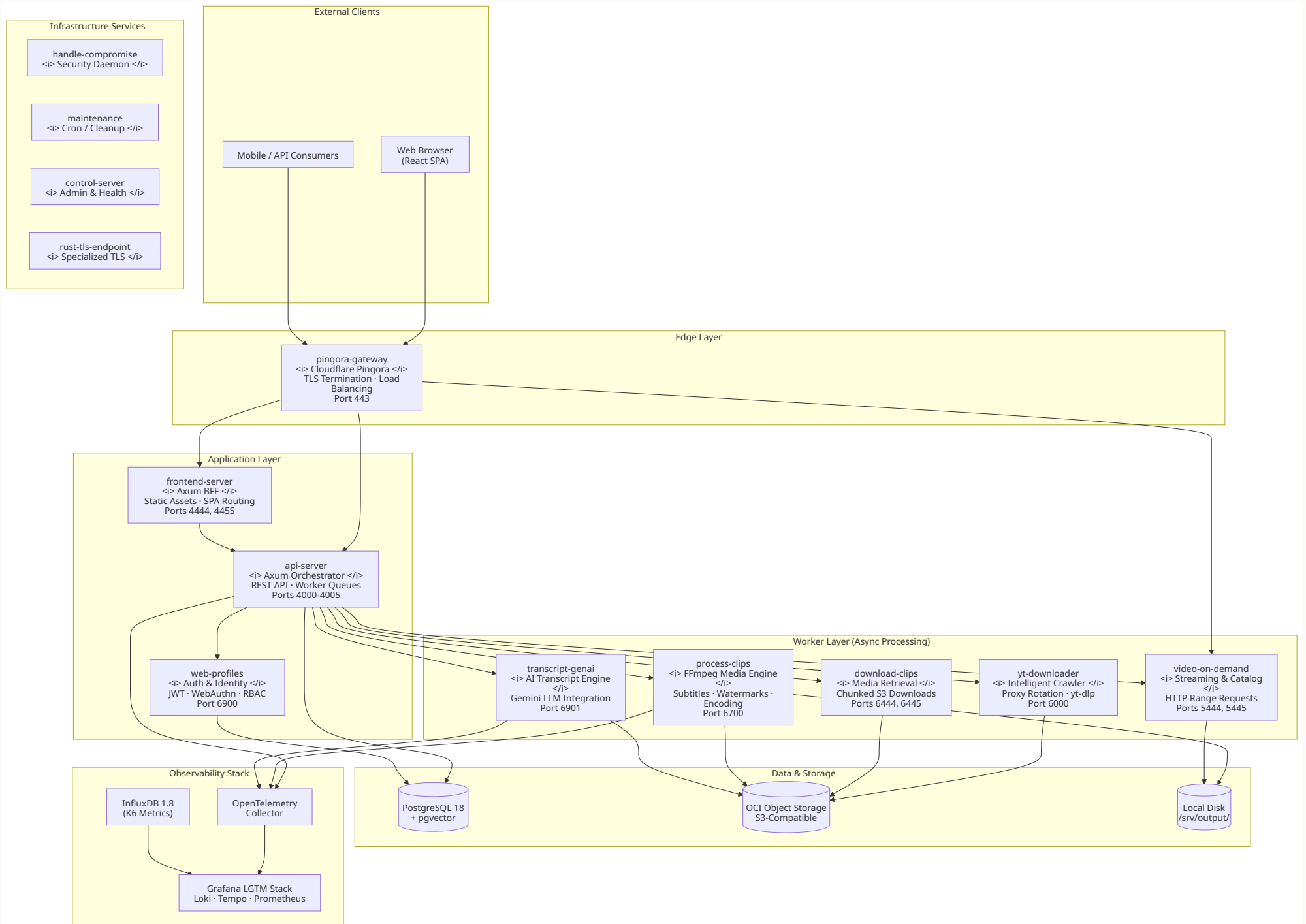


Figure 1: System Design Diagram

1.3 DevOps Pipeline & Deployment Strategy

The CI/CD pipeline is designed around a **“Build Once, Configure Everywhere”** model. Pre-compiled, statically-linked Rust binaries are committed to the repository, eliminating the need for on-runner compilation and drastically reducing deployment times.

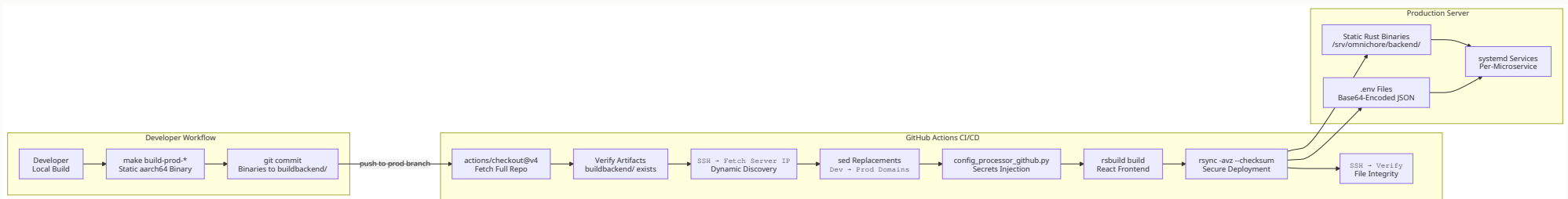


Figure 2: DevOps Pipeline & Deployment Strategy

CI/CD Pipeline Components

Stage	Technology	Purpose
Source Control	GitHub (branch-based: <code>main</code> , <code>prod</code>)	Code versioning and deployment triggers
CI Runner	GitHub Actions (<code>ubuntu-latest</code>)	Pipeline execution environment
Artifact Strategy	Pre-built static Rust binaries (<code>buildbackend/</code>)	Zero-compilation deployment
Config Injection	Python <code>config_processor_github.py</code>	Merges GitHub Secrets into JSON templates, outputs Base64 <code>.env</code> files
Domain Replacement	<code>sed</code> routines across config and JS bundles	Environment-specific domain/IP binding
Secure Transfer	<code>rsync</code> over SSH with <code>--checksum</code>	Incremental, verified file synchronization
Process Management	<code>systemd</code> unit files per service	Automatic restart, dependency ordering, resource limits
E2E Testing	Playwright (Chromium, Firefox, WebKit)	Cross-browser functional validation
Load Testing	K6 with InfluxDB + Grafana	Performance benchmarking and threshold enforcement

GitHub Actions Workflows

Workflow	Trigger	Purpose
<code>deploy-production.yml</code>	Push to <code>prod</code>	Full production deployment with config injection and rsync
<code>rust-cd.yml</code>	Push to <code>main-test</code>	Rust build verification with cargo caching
<code>playwright.yml</code>	Push/PR to <code>main-test</code>	End-to-end browser testing with artifact upload

1.4 User Journey: Video-to-Clip Pipeline

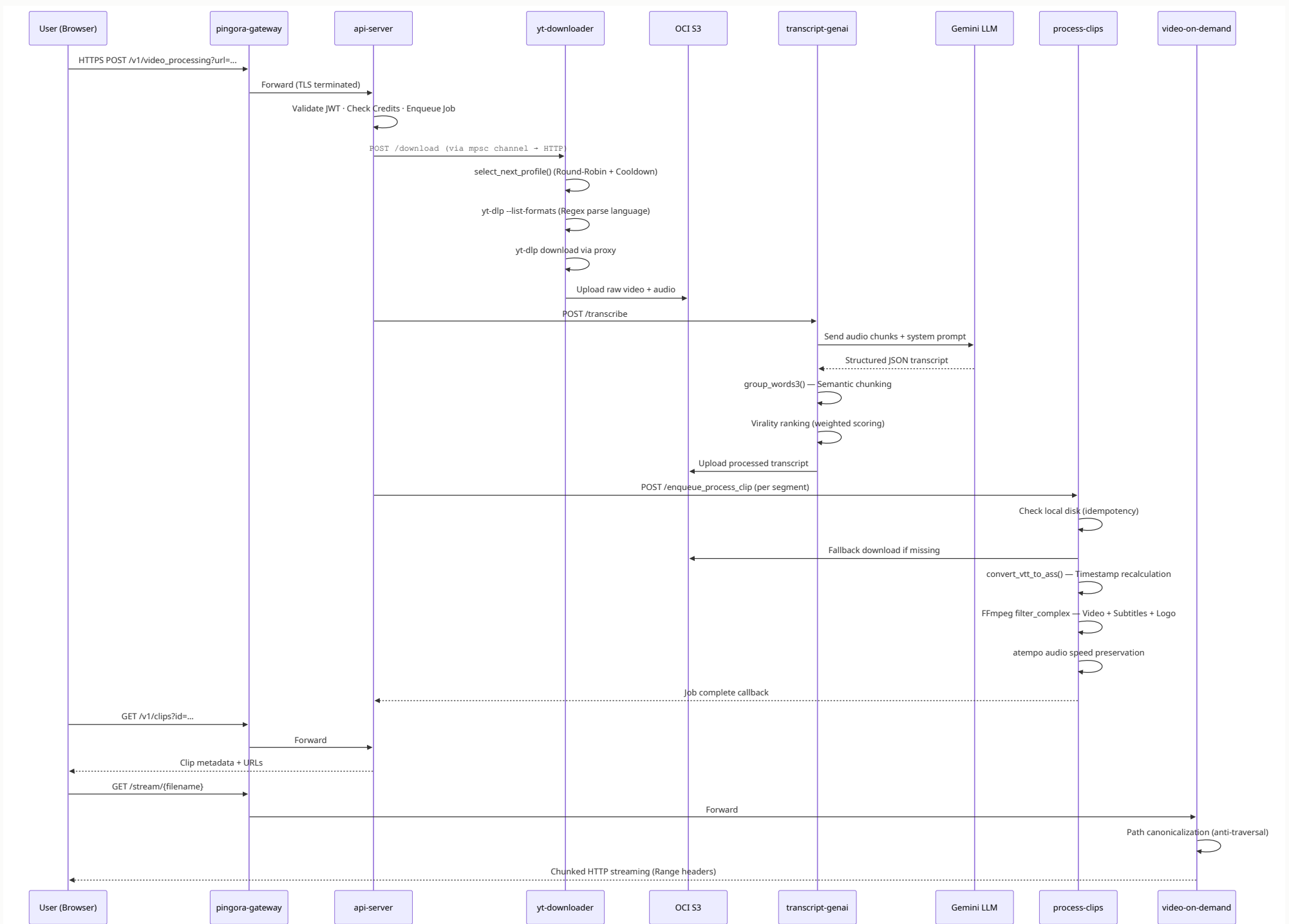


Figure 3: User Journey: Video-to-Clip Pipeline

2. Technology Stack & Microservices Breakdown

2.1 Complete Technology Matrix

Layer	Technology	Version / Detail	Rationale
Backend Language	Rust	Edition 2024 (1.85+)	Memory safety without garbage collection; zero-cost abstractions; fearless concurrency
HTTP Framework	Axum	—	Tower-based; native <code>tokio</code> integration; ergonomic extractors
Async Runtime	Tokio	—	Industry-standard async executor; multi-threaded work-stealing scheduler
API Gateway	Cloudflare Pingora	—	C10K-capable; TLS termination; connection pooling; battle-tested at Cloudflare scale
Frontend Framework	React 19	TypeScript	Component-based UI; rich ecosystem; strong typing
Frontend Build	Rsbuid	—	Rspack-powered; faster than Webpack; zero-config React support
CSS Framework	Tailwind CSS 4	—	Utility-first; design-system consistency
Routing (FE)	React Router 7	—	Declarative routing; nested layouts
Payments	Stripe	React SDK	PCI-DSS compliant payment processing
Video Streaming	HLS.js	—	HTTP Live Streaming for adaptive bitrate playback
Database	PostgreSQL 18 + pgvector	—	ACID compliance; vector similarity search; battle-tested reliability

Layer	Technology	Version / Detail	Rationale
Object Storage	OCI Object Storage (S3-compatible)	—	Cost-effective; S3 API compatibility; high durability
Cloud Provider	Oracle Cloud Infrastructure (OCI)	—	ARM Ampere instances; cost-efficient; enterprise SLAs
IaC	OpenTofu (Terraform)	OCI Provider	Declarative infrastructure provisioning; state management
Containerization	Podman	OCI-compliant	Rootless containers; Docker-compatible; daemonless architecture
Container Orchestration	Podman Compose	—	Multi-container coordination for dev/observability stack
Process Management	systemd	—	Native Linux init; automatic restart; security hardening directives
CI/CD	GitHub Actions	—	Native Git integration; matrix builds; secrets management
Media Processing	FFmpeg	—	Industry-standard transcoding; filter graphs; subtitle burning
Video Download	yt-dlp	—	Robust extractor; proxy support; format selection

Layer	Technology	Version / Detail	Rationale
AI / LLM	Google Gemini	Large + Mini multi-modal	State-of-the-art language understanding; structured JSON output
Speech-to-Text	GenAI STT	—	High-accuracy audio transcription
Observability	OpenTelemetry (OTLP)	—	Vendor-neutral traces, metrics, and logs
Monitoring	Grafana + Loki + Tempo + Prometheus	LGTM Stack	Unified dashboards; log aggregation; distributed tracing
Load Testing	K6 + InfluxDB	—	Scriptable load tests; real-time metrics; threshold-based pass/fail
E2E Testing	Playwright	—	Cross-browser (Chromium, Firefox, WebKit); CI-integrated
Encryption	AES-GCM (Vault)	—	Authenticated encryption for secrets at rest

Layer	Technology	Version / Detail	Rationale
Authentication	WebAuthn / Passkeys + JWT (EdDSA)	—	Phishing-resistant passwordless auth; compact asymmetric signatures
Serialization	serde (Rust) / JSON	—	Zero-copy deserialization; compile-time validation
Compression	Zstandard (zstd)	Level 19	Extreme compression ratios for container image distribution
Proxy Infrastructure	BrightData	—	Residential proxy rotation for rate-limit circumvention
Email	SMTP (async)	—	Transactional emails (confirmation, payment receipts)
Scraping	Python + Requests	—	Supplementary social media data extraction (X/Twitter GraphQL)

2.2 Microservices Deep Dive

The backend is a Cargo workspace comprising **17 independent crates**, each compiled into its own statically-linked binary.

pingora-gateway — Edge Gateway

Attribute	Detail
Role	Single ingress point for all external HTTPS traffic
Technology	Cloudflare Pingora (Rust)
Port	443
Capabilities	TLS termination, dynamic path-based routing, connection pooling, load balancing
Why Pingora	Written in Rust at Cloudflare for handling millions of connections; eliminates Nginx/HAProxy as external dependencies

api-server — Central Orchestrator

Attribute	Detail
Role	Central nervous system managing routing, background queues, and cross-service orchestration
Technology	Axum + Tokio + DashMap
Ports	4000–4005
Key Patterns	<code>mpsc</code> channel-based task queuing; <code>Arc<AppState></code> shared state; <code>DashMap</code> concurrent caches; <code>OnceLock</code> global config
Workers	Email sender, Stripe customer sync, video processing queue, retry handler
Auth	WebAuthn/Passkey initialization; JWT (EdDSA) issuance and validation
Why Axum	Native Tower/Tokio integration; type-safe extractors; minimal overhead

Core State Architecture:

```
AppState (Arc-wrapped)
├─ Vault (AES-GCM encryption)
├─ AppConfig (from Base64 .env → JSON)
├─ DynamicData
│   └─ public_keys (DashMap – key rotation cache)
│   └─ encrypted_secrets (DashMap – bounded user secret cache)
│   └─ revoked_tokens (DashMap – JWT revocation without DB hit)
├─ ServiceCore (20+ database repository wrappers)
├─ email_sender (mpsc::Sender<EmailTask>)
├─ stripe_sender (mpsc::Sender<StripeCustomerTask>)
└─ queue_video_processing (mpsc::Sender<QueueRecord>)
```

└─ webauthn (Arc<Webauthn>)

└─ s3_uploader (Arc<S3Client>)

process-clips — Media Processing Engine

Attribute	Detail
Role	High-performance FFmpeg wrapping engine for video manipulation
Technology	Rust + FFmpeg CLI + Axum
Port	6700
Capabilities	VTT→ASS subtitle conversion with speed-adjusted timestamps; FFmpeg complex filter graphs; watermark positioning; audio pitch preservation (<code>atempo</code>); chunked video streaming
Algorithms	<code>convert_vtt_to_ass</code> (timestamp recalculation per playback speed); <code>filter_complex</code> graph builder; path canonicalization (anti-traversal)
Why FFmpeg via CLI	Maximum codec support; hardware acceleration; proven stability for production media workloads

transcript-genai — AI Transcript Intelligence

Attribute	Detail
Role	AI-driven transcript processing, semantic word grouping, and content ranking
Technology	Rust + Google Gemini API
Port	6901
Algorithms	<code>group_words3</code> (semantic word chunking); LLM-based virality scoring; language detection; theme extraction
Why Gemini	Multimodal capabilities; structured JSON output; competitive pricing for high-volume batch processing

yt-downloader — Intelligent Video Crawler

Attribute	Detail
Role	Video/audio extraction with intelligent proxy rotation and rate-limit evasion
Technology	Rust + yt-dlp + BrightData proxies
Port	6000
Algorithms	<code>select_next_profile</code> (stateful round-robin with health checks and cooldown timers); <code>LANG_RE</code> regex parser (original audio language detection); <code>split_webm_audio</code> (FFmpeg silence-based chunking for API limits)
Why yt-dlp	Actively maintained; supports 1000+ extractors; proxy-aware; format negotiation

frontend-server — Backend-for-Frontend

Attribute	Detail
Role	Static asset serving, SPA routing, CORS enforcement, and session boundary
Technology	Rust / Axum
Ports	4444, 4455
Why Rust BFF	Eliminates Node.js runtime dependency; sub-millisecond static file serving; security header injection

web-profiles — Identity & Access Management

Attribute	Detail
Role	User accounts, profiles, authentication, authorization, billing tier management
Technology	Rust / Axum + serde
Port	6900
Capabilities	JWT issuance (EdDSA); role-based access control (RBAC); inter-service token validation; profile serialization

video-on-demand — Streaming Catalog

Attribute	Detail
Role	Finished media streaming with HTTP Range request support
Technology	Rust / Axum
Ports	5444, 5445
Capabilities	Byte-range streaming; access control via signed URLs; metadata aggregation

download-clips — Media Retrieval Worker

Attribute	Detail
Role	Asynchronous S3/cloud download worker for pre-fetching media assets
Technology	Rust + aws-sdk-s3 / reqwest
Ports	6444, 6445
Key Pattern	Idempotent downloads (filesystem metadata check before fetch); chunked streaming to prevent memory exhaustion

Infrastructure Libraries

Crate	Role
<code>oci_s3</code>	Thread-safe S3 client (<code>Arc<S3Client></code>) for OCI Object Storage; chunked upload/download; error normalization
<code>oci_command</code>	OCI REST API/CLI wrapper; PEM-based request signing; async process spawning
<code>otel</code>	Centralized OpenTelemetry initialization (<code>TracerProvider</code> , <code>MeterProvider</code> , <code>LoggerProvider</code>); service-name tagging

Operational Services

Service	Role
handle-compromise	Automated security incident response; session invalidation; key rotation; audit logging
maintenance	Data pruning; orphaned file cleanup; zombie job reconciliation; retention policy enforcement
control-server	Internal admin API; health checking; feature management
rust-tls-endpoint	Specialized TLS endpoint for mTLS / webhook callbacks
tooling	CLI utilities for data migrations, state seeding, and administrative operations

2.3 Frontend Architecture

The frontend is split into two independent React 19 applications, each built with Rsbuild:

Application	Purpose	Key Dependencies
signin (Authenticated)	Full dashboard: video processing, clip management, billing, analytics	Stripe SDK, HLS.js, Recharts, React Window, HeadlessUI
signout (Public)	Landing pages, marketing, public-facing content	Lucide icons, HeroIcons, React Router

Admin Panel (appcontrol): A separate React + TypeScript application with Tailwind CSS 4, used for internal platform administration.

3. AI Integration & Logic

3.1 AI System Design

Omnichore’s AI subsystem is architected as a **multi-stage pipeline** that transforms raw audio/video into ranked, viral-ready content segments. The AI layer is deliberately separated from the deterministic media engine (process-clips) to allow independent scaling and model evolution.

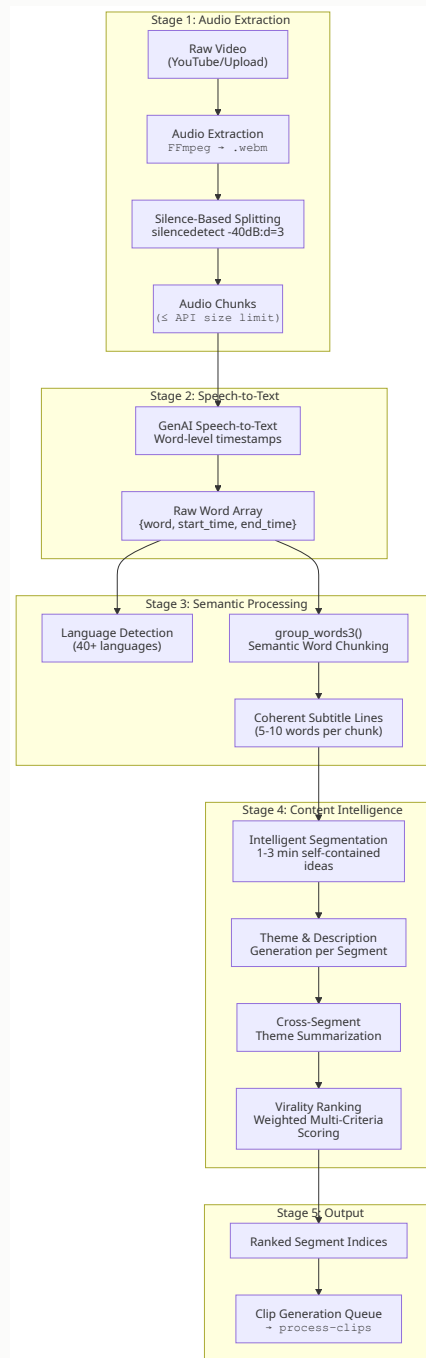


Figure 4: AI System Design

3.2 AI Balance System — Weighted Virality Scoring

The AI ranking system employs a **multi-criteria weighted scoring algorithm** to objectively rank transcript segments by their potential for social media engagement. This is not a black-box — every dimension is explicitly weighted and auditable.

Scoring Matrix

Criterion	Weight	Evaluation Dimensions
Hook Quality	30%	First 5-second attention grab; controversy potential; opening impact
Shareability	25%	Meme potential; quote-worthiness; social sharing triggers
Name Power	20%	Celebrity mentions; trending figure references; brand association
Drama Score	15%	Conflict presence; contrarian takes; debate-sparking content
Visual Proxy	10%	Imagery suggested by text; visual metaphor strength

Platform-Aware Optimization

The ranking algorithm applies platform-specific modifiers:

- **TikTok/Reels:** Prioritizes shocking openings and visual metaphors
- **Twitter/X:** Boosts debate-sparking quotes and mention-heavy segments
- **LinkedIn:** Values professional insights and statistical content

Consistency Safeguards

- **Batch normalization** against average scores
- **Recency penalty** for older cultural references
- **Diversity filter** to prevent similar-take clustering
- **Controversy cap** on polarizing content

3.3 AI Prompt Engineering System

Omnichore maintains a library of **purpose-built system prompts** stored as versioned template files, each targeting a specific stage of the AI pipeline:

Prompt Template	Purpose	LLM Target
<code>systempromptthemespriority0.txt</code>	Full segmentation + ranking for 1-3 minute viral clips with word-index preservation	Gemini Large Multimodal
<code>systempromptpriority-1.txt</code>	Lightweight virality ranking on pre-segmented summaries (Hook/Share/Name/Drama/Visual scoring)	Gemini Large Multimodal
<code>systempromptsplitclips.txt</code>	Per-segment theme + description generation	Gemini Mini Multimodal
<code>systempromptsummarize.txt</code>	Cross-segment theme consolidation and description unification	Gemini Mini Multimodal
<code>systempromptlanguage.txt</code>	Language detection engine across 40+ supported languages	Gemini Mini Multimodal

3.4 Data Communication Pipeline

All AI communication follows a strict **structured JSON protocol**:

- 1. **Input:** Raw transcript data is packaged into precisely formatted JSON with word-level indices (`"word:index"`)
- 2. **Prompt Injection:** System prompts are loaded from versioned template files and prepended to the request
- 3. **Output Parsing:** LLM responses are parsed into strongly-typed Rust structs (`TranscriptChunk` , `SegmentResult`) via `serde`
- 4. **Validation:** AI outputs are validated against expected schemas; hallucinated text is stripped; timestamps are verified against original audio timeline
- 5. **Fallback:** If AI processing fails, the system falls back to deterministic word-count-based chunking

3.5 Credit-Based AI Consumption Model

AI usage is governed by a SKU-based credit system defined in `sku.json` :

AI Provider Tier	Credit Cost per Call
Large Multimodal Model (Gemini)	2,000 credits
Mini Multimodal Model (Gemini)	1,000 credits
Speech-to-Text	500 credits

Subscription tiers control credit allocation:

Plan	Credits	Duration
Standard Subscription	1,000,000	31 days
Full Trial	250,000	7 days
Free Trial	0 (limited features)	7 days

4. Enterprise-Grade Standards & Compliance

4.1 Security Architecture

Omnichore implements **defense-in-depth** across every layer of the stack:

Authentication & Authorization

Control	Implementation
Passwordless Auth	WebAuthn / Passkeys (FIDO2) — phishing-resistant by design
Token Signing	EdDSA (Ed25519) JWT — compact, fast, quantum-resistant-ready
Token Revocation	In-memory <code>DashMap</code> of revoked JTIs — instant invalidation without database round-trips
Session Management	<code>HttpOnly; Secure; SameSite=None</code> cookies with configurable <code>Max-Age</code>
RBAC	Role-based access control enforced at the <code>web-profiles</code> service boundary
Disposable Email Blocking	800KB+ blocklist (<code>disposable_email_blocklist.conf</code>) preventing throwaway account abuse

Encryption & Secrets Management

Control	Implementation
Secrets at Rest	AES-GCM authenticated encryption via <code>Vault</code> module
Config Secrets	Base64-encoded JSON injected at deploy time via <code>config_processor_github.py</code> ; never stored in plaintext
GitHub Secrets	All sensitive values (SSH keys, API keys, database credentials) stored exclusively in GitHub Secrets
Key Rotation	<code>public_keys</code> <code>DashMap</code> supports hot-swapping rotation keys without service restart

Network & Infrastructure Security

Control	Implementation
TLS Termination	At <code>pingora-gateway</code> edge; internal services communicate over private network
Certificate Management	CSR configuration with SAN entries for all subdomains (<code>api.*</code> , <code>app.*</code> , <code>stream.*</code> , <code>download.*</code>)
Port Isolation	Each microservice bound to dedicated ports; no port sharing
Filesystem Hardening	<code>systemd</code> <code>ProtectSystem=strict</code> , <code>ProtectHome=true</code> , <code>PrivateTmp=true</code> , <code>NoNewPrivileges=true</code>
Kernel Hardening	<code>ProtectKernelTunables</code> , <code>ProtectKernelModules</code> , <code>ProtectControlGroups</code>
Network Restriction	<code>RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6</code> — blocks raw sockets
Read-Only Mounts	<code>ReadOnlyPaths</code> for shared files and templates; <code>ReadWritePaths</code> only for output directories
Capability Bounding	Gateway runs with only <code>CAP_NET_BIND_SERVICE</code> ; all other capabilities dropped
Path Traversal Prevention	<code>tokio::fs::canonicalize</code> + <code>starts_with(output_dir())</code> validation on all file-serving endpoints
Directory Traversal	Canonical path verification prevents <code>../../../../etc/passwd</code> style attacks

Incident Response

The `handle-compromise` service provides automated security response:

- **Automated session invalidation** across all services
- **Dynamic key rotation** triggered by anomaly detection
- **IP blocking** at the gateway level
- **Comprehensive audit logging** to OpenTelemetry for forensic analysis
- **Zero-trust internal authentication** — the service itself requires privileged credentials to activate

4.2 Scalability Design

Dimension	Strategy
Horizontal Scaling	Stateless microservices; shared-nothing architecture; any service can be scaled independently
Async Concurrency	<code>tokio</code> multi-threaded runtime with work-stealing; <code>DashMap</code> for lock-free concurrent caches; <code>mpsc</code> channels for backpressure-aware queuing
Connection Pooling	Pingora maintains persistent connection pools to downstream services; database connections pooled via Postgres driver
Backpressure	<code>mpsc::channel</code> with configurable buffer size (e.g., 100 items); API layer blocks when queue is full
Retry Resilience	<code>start_retry_handler</code> periodically sweeps failure queues and re-enqueues failed jobs
Idempotent Operations	Every worker checks filesystem metadata before processing; duplicate submissions produce no side effects
Static Compilation	Fat LTO (<code>lto = "fat"</code> , <code>codegen-units = 1</code>) in production profile for maximum binary optimization
Image Compression	Podman container images compressed via <code>zstd -19</code> for efficient distribution
Infrastructure as Code	OpenTofu provisions OCI ARM Ampere instances (<code>VM.Standard.A1.Flex</code>) — cost-efficient and horizontally scalable

4.3 Observability & Reliability

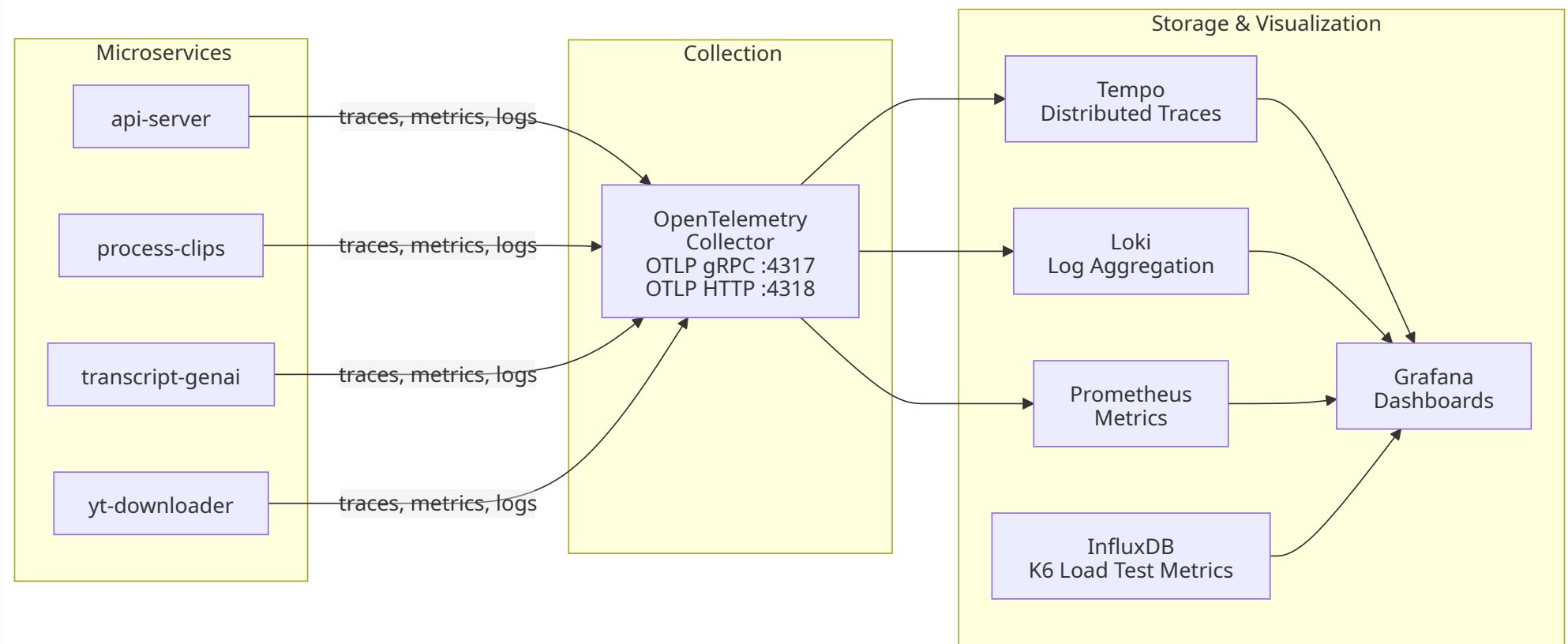


Figure 5: Observability & Reliability

Pre-Built Grafana Dashboards:

Dashboard	Purpose
<code>grafana-dashboard-prometheus.json</code>	System metrics: CPU, memory, request rates, error rates
<code>grafana-dashboard-tempo.json</code>	Distributed trace visualization across service boundaries
<code>grafana-dashboard-loki.json</code>	Centralized log search and correlation
<code>grafana-dashboard-k6.json</code>	Load test results with threshold overlays

4.4 Testing Strategy

Layer	Tool	Scope
Load Testing	K6	Multi-user concurrent video processing; 95th percentile latency thresholds (<5s); 90% success rate enforcement
E2E Browser Testing	Playwright	Chromium, Firefox, WebKit; CI-integrated with HTML report artifacts
Integration Testing	K6 endpoint scripts	Service-to-service communication validation
Security Testing	Path traversal hardening; cookie attribute validation; token revocation verification	Per-endpoint

4.5 Data Privacy & Ethical AI

Principle	Implementation
Data Minimization	User data stored only in PostgreSQL; media files have configurable retention policies enforced by <code>maintenance</code> service
Purpose Limitation	AI prompts are task-specific and stateless — no user data is retained by the LLM provider
Right to Erasure	<code>maintenance</code> service supports programmatic data pruning by user ID
Transparent AI	All AI scoring criteria are explicitly defined (Hook, Shareability, Name Power, Drama, Visual Proxy) with published weights — no opaque “black box” ranking
Anti-Bias Safeguards	Batch score normalization; diversity filters; controversy caps prevent amplification of harmful content
Language Inclusivity	System supports 40+ languages for transcript processing and content generation
Credential Hygiene	No secrets, API keys, or PII in source code; all sensitive data injected at deployment via GitHub Secrets
Audit Trail	OpenTelemetry provides full request tracing from ingress to data store for compliance auditing

4.6 Compliance-Ready Architecture

Standard	How We Align
SOC 2 Type II	Comprehensive logging (OTel); access controls (RBAC/WebAuthn); change management (Git); incident response (<code>handle-compromise</code>)
GDPR	Data minimization; retention policies; erasure capability; no cross-border LLM data retention
PCI-DSS	Payment processing delegated entirely to Stripe (PCI Level 1); no card data touches our infrastructure
OWASP Top 10	Path traversal prevention; JWT validation; CORS enforcement; input sanitization via <code>serde</code> type constraints; <code>NoNewPrivileges</code> systemd hardening

Appendix A: Repository Structure

```

omnichore/
├── backend/                                # Rust workspace (17 crates)
│   ├── Cargo.toml                         # Workspace manifest (Edition 2024)
│   └── src/
│       ├── api-server/                   # Central orchestrator
│       ├── control-server/              # Internal admin
│       ├── download-clips/              # S3 media retrieval
│       ├── frontend-server/             # Static asset BFF
│       ├── handle-compromise/           # Security response daemon
│       ├── maintenance/                 # Cron and cleanup
│       ├── oci_command/                 # OCI API adapter
│       ├── oci_s3/                      # S3 storage adapter
│       ├── otel/                        # OpenTelemetry library
│       ├── pingora-gateway/             # Edge gateway
│       ├── process-clips/               # FFmpeg media engine
│       ├── rust-tls-endpoint/           # Specialized TLS
│       ├── tooling/                     # CLI utilities
│       ├── transcript-genai/            # AI transcript engine
│       ├── video-on-demand/            # VOD streaming
│       ├── web-profiles/                # Identity & auth
│       └── yt-downloader/               # Video crawler
├── appfrontend/
│   ├── signin/                          # Authenticated React app
│   └── signout/                         # Public React app
├── appcontrol/                           # Admin panel (React + Tailwind)
├── buildbackend/                         # Pre-compiled production binaries
├── infracloud/
│   ├── podman/                          # Dockerfiles + compose.yaml
│   ├── tofu/                           # OpenTofu IaC
│   ├── production/
│   │   ├── services/                   # systemd unit files (10 services)
│   │   └── profiles/                   # Proxy rotation profiles
│   └── grafana-dashboard/              # Pre-built Grafana dashboards (4)
├── templates/                           # AI system prompts + email templates
├── sharedfiles/                         # SKU config, fonts, blocklists
├── scripts/                             # Python tooling (46 scripts)
├── scraping/                            # Social media extraction scripts
├── test-end-to-end/
│   ├── k6/                             # Load test scripts
│   └── tests/                           # Playwright E2E tests
├── .github/workflows/                   # CI/CD pipelines (4 workflows)
└── makefile                             # 726-line build automation

```

Appendix B: Port Allocation Map

Port	Service	Protocol
443	pingora-gateway	HTTPS (TLS)
4000–4005	api-server	HTTP
4444, 4455	frontend-server	HTTP
5444, 5445	video-on-demand	HTTP
6000	yt-downloader	HTTP
6444, 6445	download-clips	HTTP
6700	process-clips	HTTP
6900	web-profiles	HTTP
6901	transcript-genai	HTTP
3000	Grafana	HTTP
4317	OTLP (gRPC)	gRPC
4318	OTLP (HTTP)	HTTP
5000	PostgreSQL	TCP
8086	InfluxDB	HTTP